

Node.js + Prisma + Neon

Séance 3 - Notre projet backend prend vie

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

Node.js (suite) - npm - ESM - async/await - TypeScript - Neon - Prisma - utils/prisma.ts

Où on en est, où on va

Récap S2 et objectifs de la séance 3

Acquis de la séance 2

- Le web : client / serveur, HTTP
- JavaScript de base + notions modernes
- Node.js = JS hors navigateur
- Module http natif
- Aperçu : PostgreSQL, Neon, Prisma

Objectifs de la séance 3

- Approfondir Node.js : npm, modules, async
- Première touche de TypeScript
- Créer notre BD PostgreSQL sur Neon
- Installer et configurer Prisma
- Construire le fichier utils/prisma.ts
- Faire nos premières requêtes en BD

L'esprit de la séance. On passe de la théorie à la pratique. À la fin, vous aurez un projet Node + TypeScript connecté à une vraie base PostgreSQL hébergée sur Neon, avec Prisma comme pont. C'est dense mais on avance ensemble, étape par étape, en testant à chaque coup.

PARTIE 1

Approfondir Node.js

npm en profondeur

Initialiser un projet, installer des packages

Pour ajouter des bibliothèques à un projet Node, on utilise npm. Voici les commandes essentielles :

```
# 1) créer un projet
mkdir mon-backend && cd mon-backend
npm init -y           # crée package.json

# 2) installer des packages
npm install dotenv     # dépendance de production
npm install -D tsx      # devDependency (dev uniquement)

# 3) retirer un package
npm uninstall dotenv

# 4) installer toutes les deps (sur un clone)
npm install
```

bash

Trois choses créées

- package.json : la fiche du projet
- (noms, scripts, dépendances)
-
- node_modules/ : le dossier
- où vivent les packages installés
-
- package-lock.json : verrouille
- les versions exactes

À ajouter dans .gitignore -> node_modules/ (jamais versionner)

package.json : la carte d'identité du projet

Scripts, dépendances, configuration

```
{
  "name": "mon-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "start": "node src/index.js",
    "db:migrate": "prisma migrate dev",
    "db:studio": "prisma studio"
  },
  "dependencies": {
    "@prisma/client": "^6.0.0",
    "@prisma/adapters-neon": "^6.0.0",
    "dotenv": "^16.4.5"
  },
  "devDependencies": {
    "prisma": "^6.0.0",
    "typescript": "^5.6.0",
    "tsx": "^4.19.0"
  }
}
```

JSON

À retenir

- type: "module" : active les
- imports ES (import / export)
-
- scripts : commandes raccourcies
- (npm run dev, npm run start)
-
- dependencies : utiles en prod
- devDependencies : seulement
- pendant le développement
-
- ^x.y.z : SemVer (mises à jour
- mineures autorisées)

Les modules en JavaScript moderne

import / export (ESM)

Un module = un fichier qui partage du code avec d'autres fichiers via export / import.

math.js - exporter

```
// export nommé
export function add(a, b) {
  return a + b;
}

// export par défaut
export default function bonjour() {
  return "Salut !";
}
```

JS

app.js - importer

```
// import nommé
import { add } from "./math.js";

// import par défaut
import bonjour from "./math.js";

console.log(add(2, 3));
console.log(bonjour());
```

JS

- **Activation** : "type": "module" dans package.json
- **export nommé** : plusieurs par fichier -> import { ... }
- **export default** : un seul par fichier -> import nom from ...

Variables d'environnement avec dotenv

Ne JAMAIS coder des secrets en dur

On stocke les secrets (URL de BD, clés d'API...) dans un fichier .env hors du code et hors de Git.

Fichier .env (à la racine) :

```
DATABASE_URL="postgresql://user:pwd@  
ep-xxx.neon.tech/dbname?sslmode=require"  
  
NODE_ENV="development"
```

.env

Charger dans Node :

```
// npm install dotenv  
import dotenv from "dotenv";  
dotenv.config();  
  
console.log(process.env.DATABASE_URL);
```

JS

- Toujours ajouter .env dans .gitignore (pas dans Git !)
- Créer un **.env.example** (sans les vraies valeurs, partagé dans Git)
- `process.env.NOM_DE_VAR` pour lire une variable
- On l'utilisera juste après pour la **DATABASE_URL** de Neon

async / await en bref

Indispensable : Prisma renvoie des Promises

Les opérations qui prennent du temps (lire la BD, appeler une API) ne sont pas instantanées. JavaScript ne bloque pas le programme : il rend une Promise, et on l'attend avec await.

Sans await (PIÈGE) :

*select * from User;*

```
const users = prisma.user.findMany();  
console.log(users);  
// Promise { <pending> } -> pas la liste !
```

JS

Avec async + await :

```
async function main() {  
  const users = await prisma.user.findMany();  
  console.log(users); // la liste !  
}  
main();
```

JS

- **await** : attend qu'une Promise se termine, sans bloquer le thread
- **async** : se met devant une fonction qui contient des await
- On enveloppera la plupart de nos requêtes BD dans async/await

PARTIE 2

Un mot sur TypeScript

Pourquoi TypeScript ?

Du JavaScript avec un filet de sécurité

TypeScript = JavaScript + annotations de types. Il prévient les erreurs avant même d'exécuter le code, et améliore l'auto-complétion dans VS Code. Tout votre JS reste valide en TS.

JavaScript (typage implicite) :

```
function bonjour(nom) {  
  return `Salut ${nom}`;  
}  
bonjour(42);    // pas d'erreur visible
```

JS

TypeScript (typage explicite) :

```
function bonjour(nom: string) {  
  return `Salut ${nom}`;  
}  
bonjour(42);    // ERREUR avant l'exécution
```

TS

Pourquoi on l'introduit maintenant ?

- Notre fichier prisma.ts est en TypeScript
- Prisma génère automatiquement des types pour notre BD
- Auto-complétion magique : prisma.user.findMany() -> VS Code propose les bons champs
- Erreurs détectées avant l'exécution, énorme gain de temps

*select * from user;*

TypeScript : les bases

Types primitifs, interface, type

```
// Types primitifs
let nom: string = "Sonia";
let age: number = 25;
let actif: boolean = true;

// Tableau de string
let amis: string[] = ["Alex", "Lina"];

// Interface : la forme d'un objet
interface User {
  id: number;
  email: string;
  name?: string; // ? = optionnel
}

const u: User = { id: 1, email: "a@b.ca" };

// Fonction typée
function double(x: number): number {
  return x * 2;
}
```

TS

À retenir

- Syntaxe : variable: type
- string, number, boolean
- string[] -> tableau de string
-
- interface : décrit un objet
- type ou interface pour vos
- structures de données
-
- ? après un nom -> optionnel
-
- Une fonction peut typer ses
- paramètres ET son retour

Configurer TypeScript dans Node

TypeScript + tsx pour exécuter directement

1. Installer TypeScript et tsx (Node + TS sans étape de build) :

tsx index.ts

```
npm install -D typescript tsx @types/node
```

bash

2. Initialiser tsconfig.json (configuration TS) :

```
npx tsc --init
```

bash

3. Exécuter un fichier .ts directement avec tsx :

```
npx tsx src/index.ts  
# ou via un script : npm run dev
```

bash

tsx en une phrase

- Un outil moderne (succède à ts-node) qui exécute du TypeScript sans étape de compilation manuelle. Idéal pour le développement.

PARTIE 3

Mise en place de Neon

Créer un compte et un projet Neon

PostgreSQL prêt en 30 secondes, sans installer

Étapes (toutes dans le navigateur, plan gratuit) :

- **Aller sur** neon.tech
- **Cliquer** Sign Up (avec GitHub, Google ou e-mail)
- **Créer un nouveau projet** -> choisir une région proche (East US, par exemple)
- **Choisir le nom du projet** et la version de PostgreSQL (la plus récente)
- **Neon génère pour nous** un cluster PostgreSQL + une base par défaut

Pourquoi Neon plutôt qu'une installation locale ?

- Aucune installation à gérer (gain de temps énorme en cours)
- Accessible de partout (chez vous, en classe, en démo)
- Plan gratuit suffisant pour tous nos labos
- Adapté à l'architecture serverless (parfait avec Next.js / Vercel plus tard)

Récupérer la DATABASE_URL

Notre clé pour parler à la BD

Une fois le projet créé, Neon affiche une URL de connexion. C'est tout ce dont Prisma a besoin pour s'y connecter.

```
postgresql://USER:PASSWORD@ep-cool-name-12345.us-east-2.aws.neon.tech/neondb?sslmode=require
```

URL

On la colle dans un fichier `.env` à la racine du projet :

```
DATABASE_URL="postgresql://USER:PASSWORD@ep-cool-name.us-east-2.aws.neon.tech/neondb?sslmode=require"
```

.env

Règles d'or

- Ne JAMAIS partager cette URL (elle contient un mot de passe)
- `.env` doit être listé dans `.gitignore`
- Pour partager le format sans les secrets : créer un `.env.example`
- Si l'URL est compromise -> bouton « Reset password » côté Neon

PARTIE 4

Installer et configurer Prisma

Installer Prisma + l'adapter Neon

Trois packages à connaître

```
# Le CLI Prisma (en devDependency)
npm install -D prisma

# Le client Prisma + l'adapter Neon (en dependencies)
npm install @prisma/client @prisma/adapter-neon dotenv
```

bash

Le CLI (prisma)

- Outil en ligne de commande
- Sert à : init, migrate, generate, studio
- DevDependency (pas en production)
- Toujours via `npx prisma <commande>`

Le client (@prisma/client)

- Le code qu'on utilise dans notre app
- Dépendance de production
- Fournit `prisma.user.findMany()`, etc.
- Couplé à `@prisma/adapter-neon` pour Neon

Initialiser Prisma : prisma init

Génère `schema.prisma` et `.env`

```
npx prisma init
```

bash

```
# Crée :  
#   prisma/schema.prisma  ->  description de la BD  
#   .env                  ->  DATABASE_URL="..."
```

Le fichier `prisma/schema.prisma` (anatomie) :

```
// 1) Comment générer le client (ici, dans un dossier custom)  
generator client {  
  provider      = "prisma-client-js"  
  output        = "../generated/prisma/client"  
}  
  
// 2) Où se trouve la BD  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

prisma

Définir notre premier modèle

Le modèle User dans schema.prisma

```
// prisma/schema.prisma (suite)
```

prisma

```
model User {
  id      Int      @id @default(autoincrement())
  email    String   @unique
  name     String?
  createdAt DateTime @default(now())
  posts    Post[]
}

model Post {
  id      Int      @id @default(autoincrement())
  title    String
  content  String?
  authorId Int
  author   User     @relation(fields: [authorId], references: [id])
}
```

Lecture du modèle

- model User : une table
- Int, String, DateTime : types
-
- @id : clé primaire
- @default(autoincrement())
- @unique : valeur unique
- ? : champ optionnel
-
- Post[] : relation 1-N
- @relation : lien FK

On a décrit la BD en quelques lignes. Prisma va s'occuper de la créer pour nous.

Générer le client + créer les tables

Deux commandes magiques

1. Créer les tables dans Neon (et générer le client en même temps) :

```
npx prisma migrate dev --name init +test
```

bash

2. Si on veut régénérer le client après modification du schéma :

```
npx prisma generate
```

bash

migrate dev

- Crée un dossier prisma/migrations/
- Applique les changements à la BD Neon
- Régénère le client Prisma
- À utiliser à chaque évolution du schéma

generate

- Régénère uniquement le code client
- Dans le dossier défini par output
- Ici : ../generated/prisma/client
- Indispensable pour avoir les bons types

PARTIE 5

Le fichier `utils/prisma.ts`

Pourquoi un fichier utils/prisma.ts ?

Une instance partagée dans tout le projet

On ne veut pas créer un nouveau client Prisma à chaque fichier (chaque client ouvre des connexions à la BD). Solution : créer UNE seule instance dans un fichier central, et l'importer partout.

Si on crée un client partout

- Plusieurs connexions ouvertes
- Risque de saturer la BD
- Code dupliqué difficile à maintenir
- Configuration éparpillée

Avec un seul utils/prisma.ts

- Une seule connexion partagée
- On l'importe quand on en a besoin
- Configuration en un seul endroit
- Pattern « singleton »

Structure recommandée du projet :

```
mon-backend/  
  utils/prisma.ts    <- notre singleton (le fichier qu'on va écrire)  
  src/index.ts       <- le code qui utilise prisma  
  prisma/schema.prisma <- la description de la BD
```

tree

Par défaut, Prisma se connecte à PostgreSQL via TCP. Neon utilise une connexion serverless via HTTP/WebSocket, qui demande un client spécial. L'adapter Neon fait ce travail pour nous.

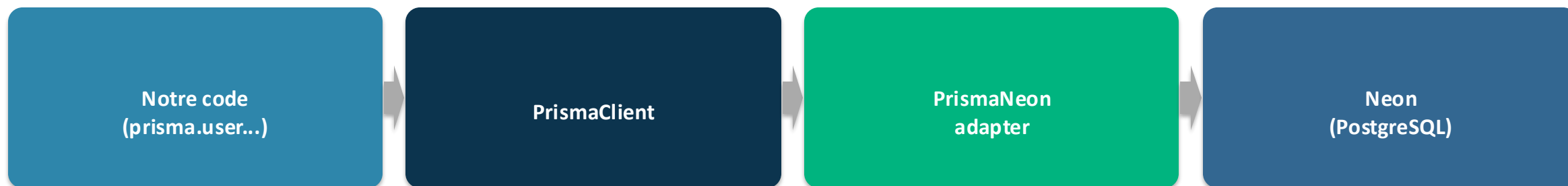
Avantages avec l'adapter Neon

- Connexions plus rapides à froid
- Compatible serverless (Vercel, etc.)
- Connexion via HTTPS (sécurisé)
- Officiellement maintenu par Prisma

Ce qu'il faut savoir

- S'installe : `npm i @prisma/adapter-neon`
- Activé via `previewFeatures`
- S'utilise dans le `PrismaClient`
- Lit la `DATABASE_URL` via `dotenv`

L'idée en un schéma :



Le fichier utils/prisma.ts (complet)

Notre singleton Prisma + adapter Neon

```
// utils/prisma.ts
```

```
import { PrismaNeon } from "@prisma/adapter-neon";
```

```
import { PrismaClient } from "../generated/prisma/client";
```

```
import dotenv from "dotenv";
```

```
dotenv.config();
```

```
// Créer le client Neon adapter
```

```
const adapter = new PrismaNeon({
```

```
  connectionString: process.env.DATABASE_URL,
```

```
});
```

```
// Créer le client Prisma avec l'adaptateur Neon
```

```
const prisma = new PrismaClient({
```

```
  adapter,
```

```
  log: ['query', 'info', 'warn', 'error'],
```

```
});
```

```
export default prisma;
```

TS

→ on utilise plateforme Neon

→ généré par Prisma

→ init pour utiliser .env

→ Connexion Neon

→ Connexion BDD - Prisma + Neon

→ Traduction sous forme JSON
→ SQL

Lecture ligne par ligne de prisma.ts

On comprend chaque morceau

Ligne(s)	Rôle
<code>import { PrismaNeon } from '@prisma/adapters-neon'</code>	Charge l'adapter qui permet à Prisma de parler à Neon (serverless)
<code>import { PrismaClient } from '../generated/prisma/client'</code>	Charge le client généré (chemin custom défini dans schema.prisma)
<code>import dotenv from 'dotenv' + dotenv.config()</code>	Lit le fichier .env -> les variables sont dispo dans process.env
<code>new PrismaNeon({ connectionString: process.env.DATABASE_URL })</code>	Crée l'adapter avec l'URL de connexion Neon (lue dans .env)
<code>new PrismaClient({ adapter, log: [...] })</code>	Crée LE client Prisma : utilise l'adapter Neon + journalise les requêtes
<code>export default prisma;</code>	Exporte l'instance UNIQUE pour qu'on l'importe ailleurs

PARTIE 6

Utiliser Prisma dans notre code

Créer une ligne en BD : prisma.user.create

Notre premier insert

```
// src/index.ts
import prisma from "../utils/prisma";

async function main() {
  const user = await prisma.user.create({
    data: {
      email: "sonia@cmaisonneuve.ca",
      name: "Sonia"
    }
  });

  console.log("Créé :", user);
}

main();
```

TS

Lancer :

```
npx tsx src/index.ts
```

bash

Lire des données : findMany & findUnique

L'équivalent du *SELECT*

Tous les utilisateurs :

```
import prisma from "../utils/prisma";

const users = await prisma.user.findMany();
console.log(users);

// Avec un filtre :
const actifs = await prisma.user.findMany({
  where: { name: "Sonia" }
});
```

TS

Un utilisateur unique :

```
// Par clé primaire
const u = await prisma.user.findUnique({
  where: { id: 1 }
});

// Par champ @unique (ici email)
const v = await prisma.user.findUnique({
  where: { email: "sonia@col.ca" }
});
```

TS

Astuce TypeScript

- Le retour est typé automatiquement (User ou null)
- Auto-complétion : tapez user. -> VS Code propose id, email, name, createdAt
- Si vous écrivez un champ inexistant : ERREUR de compilation immédiate
- C'est tout l'intérêt du combo Prisma + TypeScript

Modifier et supprimer

update et delete

Modifier (update) :

```
const updated = await prisma.user.update({
  where: { id: 1 },
  data: {
    name: "Sonia B."
  }
});

console.log("MAJ :", updated);
```

TS

Supprimer (delete) :

```
const deleted = await prisma.user.delete({
  where: { id: 1 }
});

console.log("Supprimé :", deleted);
```

TS

Le CRUD complet avec Prisma :

Action	Méthode Prisma	Équivalent SQL
Créer	prisma.user.create	INSERT
Lire	findMany / findUnique	SELECT
Modifier	prisma.user.update	UPDATE
Supprimer	prisma.user.delete	DELETE

Prisma fournit un outil graphique gratuit pour explorer la BD : voir les lignes, ajouter, modifier, supprimer, sans écrire de SQL.

```
npx prisma studio      # ouvre http://localhost:5555
```

bash

Ce que Studio permet

- Voir toutes les tables et lignes
- Ajouter une ligne via formulaire
- Modifier ou supprimer en un clic
- Filtrer, trier, chercher
- Pas besoin de SQL !

À utiliser pour

- Vérifier que notre script a marché
- Pré-remplir des données de test
- Démontrer son projet en classe
- Comprendre la structure réelle
- Débuguer en cours de développement

Récapitulatif de la séance

Ce qu'on a construit aujourd'hui

Node.js (suite)

- npm, package.json
- ESM (import/export)
- .env + dotenv
- async / await

TypeScript

- types primitifs
- interface, type
- tsx pour exécuter
- types auto avec Prisma

Neon + Prisma

- compte + projet Neon
- DATABASE_URL dans .env
- prisma init + schema
- migrate dev + generate

utils/prisma.ts

- singleton réutilisable
- adapter Neon configuré
- log des requêtes
- import partout

Les commandes à retenir :

```
npm install -D prisma typescript tsx @types/node
npm install @prisma/client @prisma/adapters-neon dotenv
npx prisma init --datasource-provider postgresql
npx prisma migrate dev --name init
npx prisma studio
npx tsx src/index.ts
```

bash

Pour la prochaine séance

À préparer + ce qu'on va voir

À faire de votre côté

- Créer un compte Neon (si pas fait)
- Cloner / créer votre projet Node + TS
- Configurer .env avec DATABASE_URL
- Lancer prisma migrate dev
- Écrire votre utils/prisma.ts
- Tester un create + findMany

Au programme ensuite

- Relations entre tables (1-N, N-N)
- Requêtes avancées : include, select, filters
- Validation des entrées (Zod)
- axios : consommer des APIs externes
- Plus tard : exposer notre BD via une API
- (Express / Next.js, plus tard)

Vous avez maintenant un vrai backend connecté à une vraie BD.

Bravo : la suite (relations, API, frontend) va se brancher dessus naturellement.

Questions ?

On a posé beaucoup de briques : prenons le temps de bien tout digérer.

Cassez, testez, corrigez - c'est comme ça qu'on progresse.